
Amazon Aurora Global Database – Complete Overview

What is Aurora Global Database?

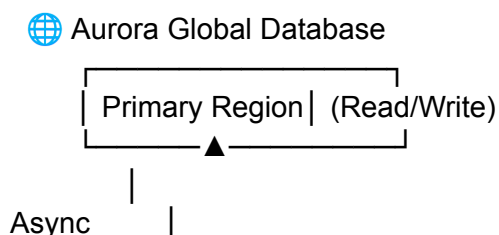
Amazon Aurora Global Database is a feature that lets you **deploy a single Aurora database across multiple AWS regions** to support:

-  **Global applications**
-  **Low-latency reads** for users around the world
-  **Disaster recovery** and high availability

 **Aurora Global** uses **asynchronous replication** to keep up-to-date copies of your database in other regions — all managed by AWS.

Key Architecture

-  One **Primary Region** — where **all writes** happen
-  One or more **Secondary Regions** — **read-only** replicas
-  **Asynchronous** replication with **~<1 second lag**



Replication ▼

Secondary 1 | (Read-only)



Secondary 2 | (Read-only)



Key Benefits

Feature	Description
 Global latency	Users read from the closest region
 Performance	Write once, read globally
 Scalability	Supports up to 5 secondary regions
 Disaster recovery	Failover to a secondary region in minutes
 Cross-region DR	Great for compliance and resilience
 Reporting & Analytics	Run heavy read queries in secondary regions
 Managed	AWS handles replication and sync



How it Works

Component	Behavior
Primary Cluster	Handles all writes and syncs data
Secondary Cluster(s)	Read-only, updated via replication
Failover	Manual to a secondary → becomes new primary

Read Latency	Sub-second in-region access
Replication Lag	~<1 sec (asynchronous)

Pricing Breakdown

You're billed for:

- Aurora compute in **each region**
- **Data transfer** between primary and secondary
- **Storage** per region
- **Backup & snapshots**

 Data replication is **billed as inter-region transfer**, so pricing varies based on regions selected.

Ideal Use Cases

Scenario	Why Use Aurora Global
 Global SaaS	Low-latency reads for global users
 Compliance / DR	Meet regional backup or availability requirements
 Read-heavy apps	Offload reads to secondaries
 Analytics	Run analytics in a secondary region without impacting production

Pros & Cons

Pros 

Cons 

Highly scalable and global Only one writable region

Fast replication (~1s)	Writes can't be distributed
Managed by AWS	Manual failover process
Seamless scaling	Extra cost for cross-region data

How to Set It Up (Console)

1.  Create an **Aurora (MySQL/PostgreSQL) Cluster**
 2.  In the **Global Database tab**, click **Create Global Database**
 3. Choose the **primary region** and **secondary regions**
 4. Aurora automatically configures **replication**
 5. Connect read apps to secondary endpoints
-

Sample Student Lab

 **Goal: Deploy a Global Aurora DB for a multi-region e-commerce app**

Steps:

1. Create Aurora cluster in **us-east-1**
2. Convert to **Global Database**
3. Add **eu-west-1** as a secondary
4. Use EC2 or Lambda in **eu-west-1** to run **read queries**
5. Simulate failover and observe metrics

 Deliverables: RTO, replication lag, read response time



Pro Tip

If your app **needs write access in multiple regions**, **Aurora Global isn't ideal**. You might consider:

-  Amazon DynamoDB Global Tables
-  Custom bi-directional replication setups (complex)



Summary Table

Feature	Aurora Global Database
Write region	1 (primary only)
Read regions	Up to 5
Replication	Asynchronous
Use Case	Read-scaling + DR
DB Engine Support	Aurora MySQL & Aurora PostgreSQL
Latency	<1 second replication lag
Failover	Manual (planned/unplanned)